PPO no LunarLander: configuração e comparação com A2C

Dupla: Gustavo Novack (novack@discente.ufg.br) e Davi de Souza Andrade (davisouza@discente.ufg.br). Fork com `algorithms/ppo.py`, `networks/discrete_actor_critic.py` e os configs usados: https://github.com/Novack-IA/t-zero-learning

Todo run usa LunarLander-v3 e duas seeds (1 e 2). As curvas mostram a média das duas seeds, com a faixa sombreada cobrindo o intervalo entre elas. As métricas foram registradas localmente, com as mesmas séries que o harness envia ao wandb; os scripts de varredura e de plotagem estão no fork, em `scripts/ppo_assignment/`. Configs: `configs/ppo_lunarlander.yml` (inicial), `configs/ppo_lunarlander_final.yml` e `configs/a2c_lunarlander.yml`.

Parte 4: resolvendo o LunarLander

O config inicial se comporta como o enunciado antecipa: sobe de −170 para cerca de +5 e para ali (`ppo_base_s1/s2`, eval −55,7 e −9,5). O config final atinge o critério nas duas seeds: `episodic_return_mean_last100` de **257,0** e **230,7** e `eval/mean_return` de **279,3** e **207,3**, dentro do orçamento de 2 milhões de passos.

Tabela de mudanças (em relação a `configs/ppo_lunarlander.yml`):

chave	antigo	novo
`total_timesteps`	1.000.000	2.000.000
`num_envs`	4	8
`ppo.num_steps`	128	256
`ppo.learning_rate`	2,5e-4	1,0e-3
`ppo.gamma`	0,99	0,999
`ppo.gae_lambda`	0,95	0,98
`ppo.update_epochs`	4	10

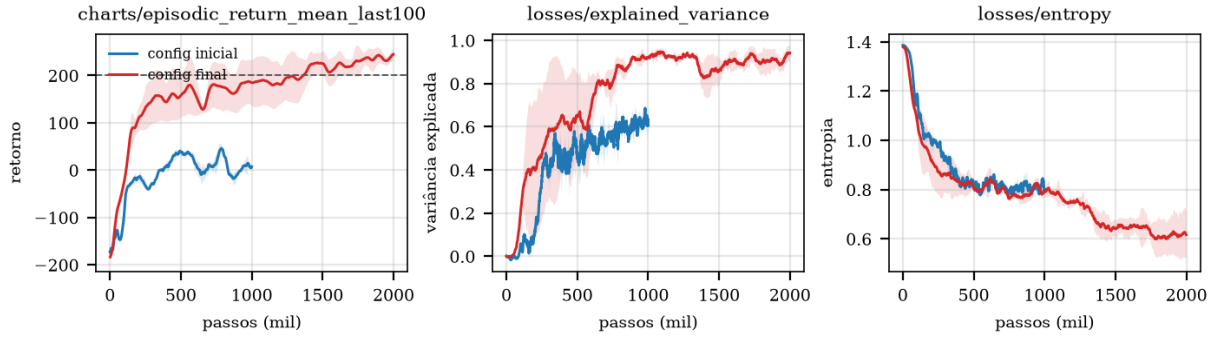


Figura 1. Config inicial (azul) e final (vermelho), 2 seeds cada. A linha tracejada é o critério de 200.

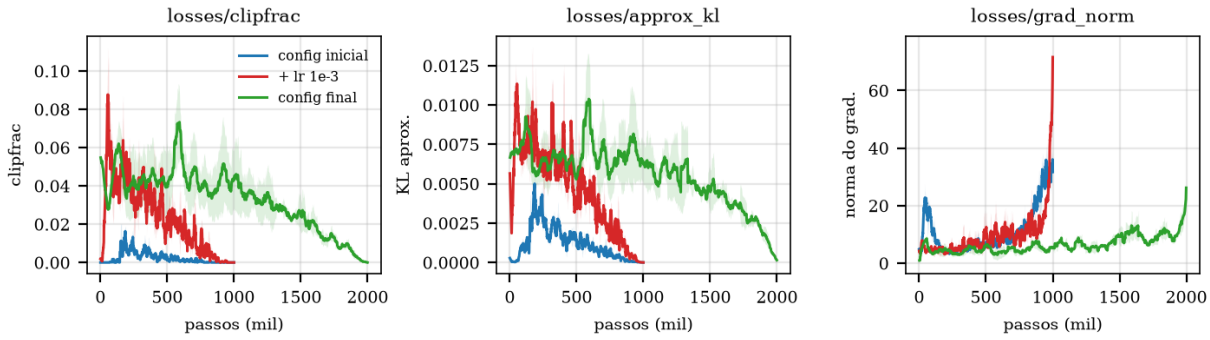


Figura 2. Diagnósticos que motivaram as mudanças: config inicial (azul), só com learning_rate = 1e-3 (vermelho) e config final (verde).

learning_rate 2,5e-4 → 1,0e-3. O motivo está na Figura 2: no config inicial o `clipfrac` fica em torno de 0,004 e o `approx_kl` em 0,0018, ou seja, em 99,6 por cento das amostras o clipping nunca chega a ser acionado. O mecanismo central do PPO, a região de confiança, nunca é o fator limitante; a política simplesmente quase não se move entre um rollout e o seguinte, e as dez épocas de reuso devolvem quase o mesmo gradiente. Elevar a taxa para 1e-3 (curva vermelha) leva o `clipfrac` a 0,030 e o `approx_kl` a 0,0058, e sozinha essa mudança tira o retorno de ~5 para 176,9 e 113,3, com eval 187,9 e 143,6. Foi a mudança de maior efeito isolado de toda a busca.

update_epochs 4 → 10. Consequência direta da leitura anterior. Com o `clipfrac` ainda em 0,03 depois do ajuste da taxa, havia folga na região de confiança, e é exatamente para isso que o clipping existe: tornar seguro reusar o mesmo rollout mais vezes. Na Figura 2 a curva verde mantém `clipfrac` entre 0,04 e 0,05 ao longo de quase todo o treino, isto é, a restrição passou a estar ativa de forma sustentada, e não apenas em picos. Esta foi a diferença decisiva entre a penúltima e a última configuração: um candidato idêntico ao final mas com `update_epochs=4` (`c1`, `num_envs=16`, `num_steps=128`, mesmo batch de 2048) chegou a 201,8 e 241,2 de retorno de treino, porém falhou no critério de avaliação com eval 151,0 e 171,1. A entropia acompanha a diferença: `c1` termina entre 0,85 e 0,90, enquanto o config final desce a 0,62 (Figura 1, painel direito), e como a avaliação é estocástica uma política ainda dispersa perde pontos de combustível e precisão de pouso. Mas a entropia é sintoma, não causa, e testamos isso: baixar `ent_coef` de 0,01 para 0,001 e para 0,0 sobre o `c1`, em vez de aumentar o reuso, piorou o resultado (61,1 e 119,8 com `ent_coef=0,001`; 178,8 e 207,4 com 0,0, ainda sem passar nas duas seeds). Forçar a política a ficar determinista não substitui aprender melhor; a entropia menor do config final é consequência das dez épocas de reuso, não de uma penalidade maior.

num_envs 4 → 8 e num_steps 128 → 256. O batch sobe de 512 para 2048 passos. Um episódio de LunarLander dura centenas de passos, então um rollout de 512 passos distribuído em 4 ambientes raramente contém um episódio completo: quase toda a vantagem vinha do bootstrap do crítico, e não de recompensa observada. O sintoma está na Figura 1, painel do meio: a `explained_variance` do config inicial estaciona em 0,6, enquanto o config final chega a 0,9. Com um crítico ruim e vantagens quase inteiramente bootstrapadas, aumentar

o reuso de amostras só amplificaria o erro do crítico.

gamma 0,99 → 0,999 e gae_lambda 0,95 → 0,98. Com gamma 0,99 o horizonte efetivo é de cerca de 100 passos, e o bônus de +100 por pousar chega ao fim de um episódio de até 1000 passos, logo descontado a um valor desprezível no início da descida. O que torna esta mudança interessante é que ela **não funciona sozinha**: testada isoladamente sobre o config inicial, o retorno ficou em 13,8 e −58,8 e a `explained_variance` desabou para 0,001. O horizonte mais longo torna o alvo do crítico muito mais difícil, então ela só compensa acompanhada do batch maior e da taxa de aprendizado maior, que dão ao crítico condição de acompanhar. O `gae_lambda` subiu junto pela mesma razão: com horizonte longo, convém apoiar-se mais em recompensa observada e menos no bootstrap.

total_timesteps 1M → 2M. A Figura 1 mostra que o config final cruza 200 só por volta de 850 mil passos e continua subindo até o fim; com 1 milhão de passos ele terminaria perto de 190, abaixo do critério. O orçamento permite 2 milhões.

Runs que não deram certo e o que ensinaram. (i) `max_grad_norm` 0,5 → 5,0: a `grad_norm` do config inicial ficava entre 5 e 7 contra um teto de 0,5 (Figura 2, painel direito), o que sugeria que o corte de gradiente era o freio. O retorno piorou, para −36,3 e −43,9. Lido junto com o `clipfrac` quase nulo, o diagnóstico correto aparece: o gradiente era grande, mas a política mesmo assim não se movia, então o freio era o tamanho do passo, não o corte. (ii) `num_steps` 128 → 512 sozinho: −39,9 e −27,3, com `explained_variance` de 0,1. Um batch maior com a mesma taxa minúscula significa apenas menos atualizações, igualmente pequenas. (iii) Um candidato no estilo do SB3 zoo (`num_steps=1024`, 32 minibatches, lr 3e-4) ficou em 121,1 e 111,7: poucas iterações para o orçamento disponível. (iv) As duas reduções de `ent_coef` descritas acima, que levaram à leitura corrigida sobre o papel da entropia.

Parte 5: PPO contra A2C

O A2C é o `algorithms/a2c.py` da atividade anterior, com o mesmo orçamento de 2 milhões de passos. A primeira tentativa, com os valores do CartPole (`num_steps=5`, lr 7e-4, gamma 0,99), terminou em −108,3 e −102,8; a tentativa usada aqui ajusta `num_steps` para 32, lr para 3e-4 e gamma para 0,999, e chega a −41,2 e −23,4 (eval −54,7 e 18,0).

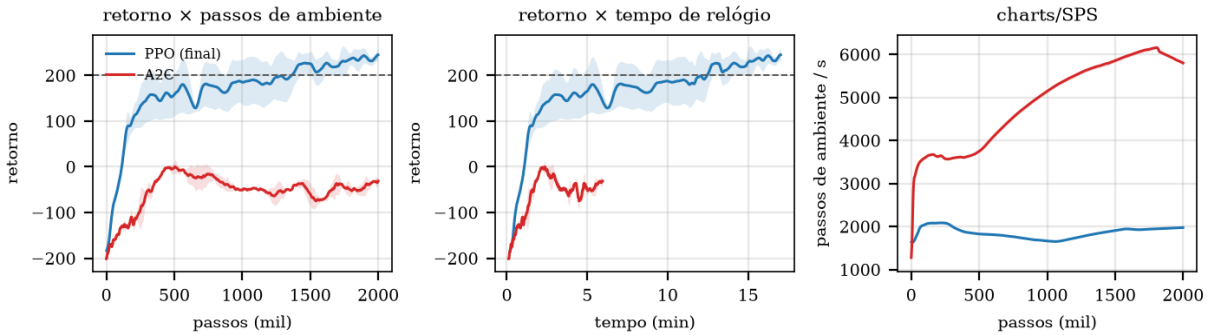


Figura 3. PPO final (azul) contra A2C (vermelho), 2 seeds cada: retorno por passos de ambiente, retorno por tempo de relógio e vazão.

Eficiência amostral. O PPO chega a retorno 0 em cerca de 100 mil passos, a 100 em 210 mil e a 200 em 853 mil. O A2C chega a 0 apenas em 518 mil passos e nunca alcança 100 nem 200, estacionando entre −20 e −40. Com o mesmo orçamento de passos de ambiente, a diferença não é de grau: um resolve o ambiente e o outro não sai do lugar.

Tempo de relógio e vazão. O A2C é mais rápido por passo de ambiente, com `charts/SPS` em torno de 6000 contra cerca de 2000 do PPO, e completa os 2 milhões de passos em cerca de 6 minutos contra 17 do PPO. A explicação está na contagem de atualizações: o PPO faz 976 iterações × 10 épocas × 4 minibatches = 39.040 passos de gradiente, enquanto o A2C faz 2.000.000/(16 × 32) = 3.906. São dez vezes mais passos de gradiente, cada um sobre um minibatch de 512 amostras, para a mesma quantidade de interação com o ambiente. Ressalva honesta: estes runs foram executados em paralelo na mesma máquina, então os valores absolutos de SPS e de tempo total estão contaminados por disputa de CPU; a razão entre os dois, porém, é consistente com a contagem de atualizações acima.

Mesmo orçamento. Com os mesmos passos de ambiente o PPO vai muito mais longe. Trocar o orçamento para tempo de relógio não muda a resposta: no painel do meio da Figura 3, aos 6 minutos (todo o tempo que o A2C levou para gastar os 2 milhões de passos) o PPO já está perto de 180, enquanto o melhor ponto do A2C em toda a sua execução foi cerca de 0. O PPO ganha nos dois eixos, ainda que a vantagem em tempo seja menor que em passos.

Estabilidade. A faixa entre as seeds do A2C é estreita, mas isso só significa que as duas falham de forma parecida. O PPO tem faixa larga no meio do treino, entre cerca de 100 e 250, e uma queda visível por volta de 650 mil passos, mas as duas seeds terminam acima de 200. A queda coincide com o pico de `clipfrac` em 0,06 na Figura 2, isto é, com o momento em que as atualizações mais empurraram a política para fora da região de confiança.

Mecanismo. Cada diferença observada aponta para uma das quatro mudanças do PPO. A diferença de eficiência amostral vem do **reuso de amostras** combinado com a **razão de probabilidades** e o **clipping**: são os três juntos que permitem dez passagens sobre o mesmo rollout, já que a razão corrige o fato de os dados terem sido coletados por outra política e o clipping impede que essas passagens afastem demais a política dos dados. O `clipfrac` sustentado em 0,04 a 0,05 na Figura 2 é a evidência de que essa restrição esteve ativa, e não decorativa; sem ela as dez épocas seriam atualizações fora da política sem nenhuma correção. A diferença de qualidade do crítico vem do **GAE** e do batch maior: a `explained_variance` do PPO fica em 0,9 (Figura 1) contra praticamente zero no A2C (−0,002 e 0,025), porque o retorno de 32 passos do A2C com gamma 0,999 é quase inteiramente bootstrap sobre um crítico que ainda não aprendeu, enquanto o GAE com lambda 0,98 apoia a estimativa majoritariamente em recompensa observada. Por fim, a vantagem do A2C em vazão é o preço do reuso: ele é mais barato por passo de ambiente exatamente porque faz um décimo dos passos de gradiente.